

User Plugin

Solves: **user.management** · Namespace: **Plugins\User** · Type: on-demand GDA module

The User plugin owns the **user.management** business domain on the AlfacodeTeam PhpServicePlatform (GDA) framework. It provides enterprise-grade user registration, lookup, partial update, email verification, soft-deletion, and **timing-safe, rate-limited credential verification** — all over a multi-tenant `users` table.

It is the canonical reference for how a first-party plugin is structured: pure Domain, an Application service that owns the transaction + events, Infrastructure adapters behind ports, and a published API contract that other modules consume.

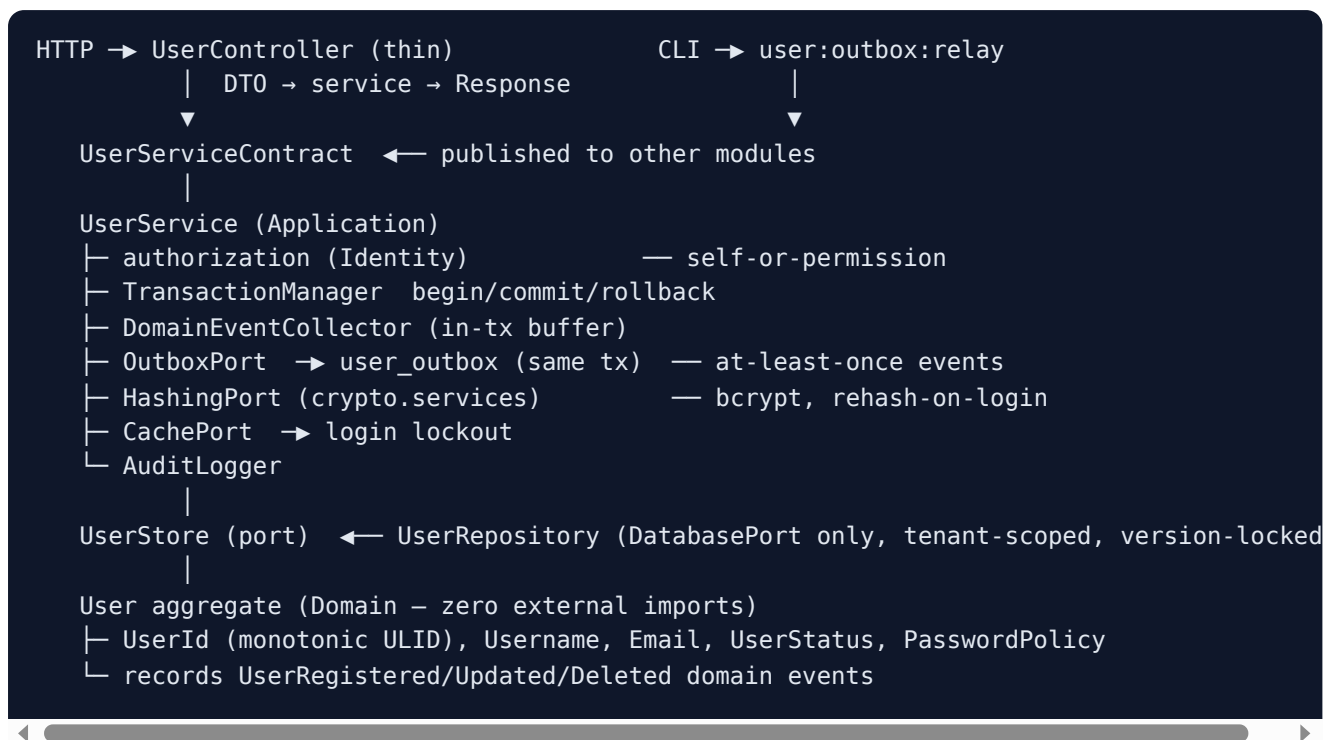
Table of contents

1. [What it does](#)
 2. [Architecture at a glance](#)
 3. [Directory layout](#)
 4. [Data model](#)
 5. [HTTP API](#)
 6. [Web UI \(AJAX + CSRF\)](#)
 7. [Security model](#)
 8. [Reliability: the transactional outbox](#)
 9. [Installation & wiring](#)
 10. [Configuration](#)
 11. [Using it from another plugin](#)
 12. [Using it from a project](#)
 13. [CLI](#)
 14. [Testing](#)
 15. [Extending the pattern](#)
-

What it does

Capability	Entry point	Notes
Register a user	POST /api/users	Public, rate-limited; emits <code>user.registered</code>
List users	GET /api/users	Admin-only; keyset paginated
Show a user	GET /api/users/{id}	Self or <code>user:read-any</code>
Update (partial)	PUT/PATCH /api/users/{id}	Self or <code>user:update-any</code> ; optimistic-locked; emits <code>user.updated</code>
Verify email	POST /api/users/{id}/verify-email	Pending → Active; emits <code>user.updated</code>
Soft-delete	DELETE /api/users/{id}	Self or <code>user:delete-any</code> ; emits <code>user.deleted</code>
Verify credentials	<code>UserServiceContract::verifyCredentials()</code>	Timing-safe, lockout, rehash-on-login (used by an Auth module)
HTML UI	GET /users , /users/create , /users/{id} , /users/{id}/edit	AJAX-driven, cookie auth, CSRF on every form

Architecture at a glance



The five GDA access rules hold: Controller → Service (contract only), Service → Repository + Gateway, Repository → DatabasePort only, Domain imports nothing external.

Directory layout

```
plugins/User/
├─ module.json                single source of truth (routes, requires, config)
├─ Provider.php              DI wiring + CLI registration
├─ API/
│   └─ Contracts/UserServiceContract.php    published interface
│       └─ DTOs/
│           └─ IntegrationEvents/
│               Register/Update/VerifyEmail/User/ListUse
│               UserRegistered/Updated/Deleted + Generic
├─ Application/
│   └─ Contracts? (none)
│       └─ Ports/ UserStore.php, OutboxPort.php    internal DIP seams (testability)
│           └─ Services/UserService.php            transaction + events + authz + lockout
├─ Domain/
│   └─ Entities/User.php        aggregate, private ctor, named construct
│       └─ Events/
│           UserRegistered/Updated/Deleted domain e
│       └─ Exceptions/DuplicateUserException.php
│           └─ ValueObjects/  UserId, Ulid, Username, Email, UserStatus, PasswordPolicy
├─ Infrastructure/
│   └─ Audit/AuditLogger.php
│       └─ Cli/RelayUserOutboxCommand.php        user:outbox:relay
│           └─ Http/Controllers/  UserController, UserPageController
│               └─ Outbox/  OutboxWriter, OutboxRelay
│                   └─ Persistence/UserRepository.php    DatabasePort only
├─ config/user.php
├─ database/
│   └─ migrations/  create_user_table, create_user_outbox_table
│       └─ seeders/UserSeeder.php
│           └─ factories/UserFactory.php
├─ resources/views/  layouts/app.php, users/{index,create,edit,show}.php
```

Data model

`users` table (owned by the migration — the repository never alters schema):

Column	Type	Purpose
<code>id</code>	bigint PK	internal surrogate (never leaves persistence)
<code>tenant_id</code>	char(31)	owning tenant (' ' = single-tenant)
<code>user_id</code>	char(31)	public ULID identifier
<code>username</code>	varchar(50)	unique per tenant
<code>email</code>	varchar(150)	unique per tenant, lowercased
<code>password_hash</code>	char(60)	bcrypt (exactly 60 chars)
<code>remember_token</code>	char(64) null	SHA-256 of the remember-me token
<code>status</code>	tinyint	1=active, 2=inactive, 3=pending
<code>version</code>	int unsigned	optimistic-lock version
<code>email_verified_at</code>	timestamp null	set on confirmation
<code>created_at</code> / <code>updated_at</code> / <code>deleted_at</code>	timestamps	soft-delete aware

Uniqueness is **per tenant** (`uniq_tenant_username` , `uniq_tenant_email`).

A second table, `user_outbox` , stores integration events for reliable delivery.

HTTP API

All API responses use the framework envelope:

```
// success
{ "data": { "id": "01J...", "username": "jane", "email": "jane@example.com",
            "status": "active", "emailVerified": true, "createdAt": "2026-..." } }

// list (keyset paginated)
{ "data": [ ... ], "meta": { "count": 25, "limit": 25, "has_more": true,
                           "next_cursor": "01J..." } }

// error
{ "error": { "code": "...", "message": "...", "fields": { "email": "..." } } }
```

Register

```
curl -X POST https://app.example.com/api/users \
  -H 'Content-Type: application/json' \
  -d '{"username":"jane","email":"jane@example.com","password":"C0rrectHorse!"}'
# 201 → { "data": { ... } }   emits user.registered
```

List (paginate)

```
curl 'https://app.example.com/api/users?limit=50&after=01J...' \
-H 'Authorization: Bearer <token>' # or same-site session cookie
```

Update (partial / PATCH semantics)

```
curl -X PUT https://app.example.com/api/users/01J... \
-H 'Content-Type: application/json' -H 'X-CSRF-Token: ...' \
-d '{"email":"new@example.com"}' # only changed fields; bumps version
```

A concurrent edit that loses the version race → **HTTP 409** (OptimisticLock). A duplicate username/email → **HTTP 409/422** (DuplicateUserException).

Web UI (AJAX + CSRF)

`UserPageController` renders four pages (`/users` , `/users/create` , `/users/{id}` , `/users/{id}/edit`). Each is a thin HTML shell that hydrates over AJAX against `/api/users` . Authentication is **same-site cookie** (no bearer token in the browser).

CSRF on every form: the page controller (via `ViewController` → `InteractsWithCsrf`) mints an HMAC token bound to a dedicated `csrf_bind` cookie. Each page exposes it as `<meta name="csrf-token">` and a hidden `_csrf_token` field; the shared `window.UserApp` client sends it as the `X-CSRF-Token` header on every unsafe (POST/PUT/PATCH/DELETE) request.

Project requirement: wire a `CsrfTokenLayer` in `withSecurity()` with `bindCookie: 'csrf_bind'` , the same lifetime as `CSRF_LIFETIME` , and **do not exempt** `/api` (the UI authenticates by cookie, so the write endpoints must be CSRF-checked). Otherwise tokens are sent but never validated.

Security model

Concern	Mechanism
Authorization	In the service : admins act on anyone (<code>user:list</code> , <code>user:*-any</code>); a non-admin only on their own record (<code>hash_equals</code> self-check)
Password storage	<code>HashingPort</code> (bcrypt); never <code>password_hash()</code> directly; plaintext never persisted/logged/returned
Password strength	<code>PasswordPolicy</code> VO — length 12–72, ≥3 char classes, deny-list
Login timing	Constant-time decoy hash for unknown users
Brute force	Per-identifier lockout via <code>CachePort</code> (5 failures / 15 min)
Hash upgrades	<code>needsRehash()</code> → transparent rehash on successful login
Multi-tenancy	Every query scoped by <code>Identity->tenantId</code>
PII in logs	Exception context carries IDs only; audit pseudonymises identifiers
Credentials in transit	<code>password_hash</code> / <code>remember_token</code> never appear in any DTO/JSON
Audit	<code>AuditLogger</code> writes structured JSON for register/update/delete/login/lockout/rehash

Reliability: the transactional outbox

Integration events are **not** dispatched directly after commit (which can be lost on a crash). Instead:

1. The service writes the event into `user_outbox` **inside the same transaction** as the state change (atomic).
2. `user:outbox:relay` (cron/supervised) reads pending rows and dispatches them to the `EventBus` — **at-least-once**, idempotency keyed by the event UUID.

```
register/update/delete —tx→ [users row + user_outbox row] COMMIT
                        |
cron: php cli user:outbox:relay → EventBus → subscribers
```

Consumers must dedupe on the event id (it may be redelivered after a crash between dispatch and mark-dispatched).

Installation & wiring

1. **Enable the plugin** (publishes config/, database/, resources/):

```
bash hkm plugins enable User
```

2. **Register the Provider** in your project bootstrap:

```
php // projects/<name>/bootstrap/app.php return $builder ->withModules([
Plugins\Crypto\Provider::class, // crypto.services (HashingPort)
Plugins\View\Provider::class, // view.rendering
Plugins\User\Provider::class, // user.management ]) ->build();
```

3. **Ensure required capabilities are available:** `database.query` (DatabasePort), `crypto.services` (HashingPort), `cache.redis` (CachePort), `view.rendering` (ViewRendererContract). The plugin declares these in `requires[]`, so boot fails fast if one is missing.

4. **Run migrations:**

```
bash php cli migrate:run php cli db:seed --class=UserSeeder # optional baseline
admin
```

5. **Schedule the outbox relay** (e.g. cron every minute):

```
cron * * * * * cd /app && php cli user:outbox:relay --limit=500
```

Configuration

`.env` keys (all optional):

```
HASH_BCRYPT_COST=12      # bcrypt cost (crypto.services)
CSRF_BIND_COOKIE=csrf_bind # must match the CsrfTokenLayer bindCookie
CSRF_LIFETIME=43200      # must match the CsrfTokenLayer lifetime (seconds)
```

`module.json` declares `requires`, `routes`, `views`, `emits`, `commands`, and `config[]`. Every env var the plugin reads is declared in `config[]` (boot fails otherwise).

Using it from another plugin

Depend on the **published contract**, never on internals. Declare the domain in your `module.json`:


```
// plugins/Billing/module.json
{ "requires": ["user.management", "database.query"] }
```

Inject the contract in your Provider:

```
use Plugins\User\API\Contracts\UserServiceContract;

$container->bind(InvoiceService::class, fn($c) => new InvoiceService(
    users: $c->make(UserServiceContract::class), // resolvable because you require user
));
```

```
final class InvoiceService
{
    public function __construct(private readonly UserServiceContract $users) {}

    public function billFor(string $userId): void
    {
        $user = $this->users->find($userId); // ?UserDTO – primitives only
        if ($user === null) { /* ... */ }
    }
}
```

React to user lifecycle events instead of polling — subscribe in `boot()`:

```
public function boot(HttpPipeline $http, CliPipeline $cli, WorkerPipeline $w, EventBus $e)
{
    $events->subscribe('user.registered', SendWelcomeEmailListener::class);
    $events->subscribe('user.deleted', PurgeUserDataListener::class);
}
```

The event payload is **primitives only** (`userId`, `username`, `email`, `occurredAt`, `version`) — your module never needs the User plugin's value objects.

Scope isolation still applies: requiring `user.management` grants the **public contract** only. `UserRepository` and other `bindInternal` bindings throw `ScopeViolationException` if resolved cross-scope.

Using it from a project

A project route can opt into the plugin without loading it app-wide:

```
// projects/<name>/proj.json
{ "routes": [
  { "method": "GET", "path": "/admin/users",
    "handler": "Projects\\Admin\\Http\\AdminUserController@index",
    "requires": ["user.management"], "filters": ["auth"] }
] }
```

```
namespace Projects\Admin\Http;

use Plugins\User\API\Contracts\UserServiceContract;
use Plugins\User\API\DTOs\ListUsersQuery;
use Project\Http\Controllers\ApiController;

final class AdminUserController extends ApiController
{
    public function __construct(private readonly UserServiceContract $users) {}

    public function index(): Response
    {
        $page = $this->users->list(ListUsersQuery::fromRequest($this->resolveRequest()));
        return $this->paginated(array_map(fn($u) => $u->toArray(), $page->items),
                                total: count($page->items), page: 1, perPage: $page->limit);
    }
}
```

Project views override plugin views by default (project-first cascade); target a specific plugin view with `user::users/index`.

CLI

```
php cli user:outbox:relay          # relay up to 100 pending events
php cli user:outbox:relay --limit=500
```

Returns the number dispatched. Idempotent and safe to run concurrently (rows are claimed via a status guard).

Testing

The service depends on the `UserStore` and `OutboxPort` interfaces (DIP), so it is fully unit-testable with in-memory fakes — no database:

```
$svc = new UserService(
    repository: new FakeUserStore(),
    transaction: new TransactionManager(new FakeDatabasePort()),
    collector: new DomainEventCollector(),
    outbox: new FakeOutbox(),
    hasher: new FakeHasher(),
    identity: Identity::asAdmin('tenant-1'),
    cache: new FakeCache(),
    audit: new AuditLogger('tenant-1', 'admin', fn($l) => null),
);
```

See [tests/Unit/Plugins/User/](#) (13 tests): registration, duplicate rejection, weak-password rejection, authorization (self vs admin), update/delete event emission, login lockout, and rehash-on-login. Run:

```
vendor/bin/phpunit tests/Unit/Plugins/User
```

Extending the pattern

This plugin is a template. To build your own domain module the same way:

1. **module.json** — declare `solves`, `requires`, `exposes`, `routes`, `emits`, `config`. One module, one domain.
2. **Domain** — `final` entity with a **private constructor** + named constructors (`create` / `reconstitute`); value objects for every concept; record domain events in the aggregate. Zero external imports.
3. **API** — a published `...ServiceContract` interface + DTOs (validation in `fromRequest`) + primitives-only integration events.
4. **Application** — a service that owns the `TransactionManager`, collects domain events, writes integration events to an **outbox** in-tx, and does **authorization first**.
5. **Infrastructure** — repository (DatabasePort only, tenant-scoped, optimistic-locked), gateways (vendor SDK only), thin controllers (≤ 3 lines). Hide concretes behind **internal ports** so the service stays testable.
6. **Provider** — `register()` binds internals with `bindInternal` and the contract with `bind`; `boot()` registers hooks / CLI commands / event subscriptions.

Copy [plugins/User/](#) as a starting skeleton, rename the namespace, and replace the domain.

Part of the *AlfacodeTeam PhpServicePlatform*. See the root [CLAUDE.md](#) and [docs/ai-context/](#) for framework-wide architecture.